ThOR: Threat Obfuscation (re)Routing for Network-on-Chip

Aidan Mulvaney, Duncan Zhang, Sarannya Bhattacharya, and Zequan Tang

University of Wisconsin–Madison

Abstract

As System-on-Chip (SoC) designs grow in complexity with multiprocessing and heterogeneous computing, integrating third-party hardware intellectual property (IP) has become a standard practice to accelerate development. However, this reliance on external IP introduces significant security vulnerabilities. Since a Network-on-Chip (NoC) connects multiple processors, the NoC is an attractive target for hardware Trojans (HT), malicious modifications at the hardware level that compromise data integrity and waste system resources. Designing robust and secure measures analogous to traditional network security is essential to mitigate HTs. This paper proposes a pseudo-random routing algorithm that dynamically changes the edge weights between routers, which may cause two identical transactions to take different paths. By changing the edge weights on the fly, a packet in the network may be rerouted at each step, obfuscating the path from attackers. By leveraging pseudo-random network rerouting, we aim to mitigate HTs with high probability while introducing acceptable area and power overheads and scale efficiently to larger NoCs.

1. Introduction

Hardware Trojans (HTs) are malicious circuits inserted into NoC routers, often at input ports, to intercept, corrupt, or delay packets [2]. Undetectable during manufacturing and testing, they pose runtime risks, including information leakage and performance degradation. Traditional runtime mitigation strategies, like detecting and isolating infected routers via anomaly detection units, can reroute traffic around compromised components [1, 3]. Still, these solutions introduce significant hardware and computation overhead that degrade network performance. Thus, a more efficient and lightweight solution is needed.

In this work, we focus on a threat model where HTs not only increase packet latency, modeling potential data snooping, but also intentionally misroute packets away from their destinations to disrupt network traffic. Such behaviors exacerbate congestion and further degrade performance. These challenges are particularly problematic for simple, routing algorithms such as X-Y Dimension-Order Routing (DOR).

X-Y DOR, widely adopted for its simplicity and low overhead, routes packets first along the X-axis and then along the Y-axis. Figure 1a demonstrates the typical routing path from router 12 to router 3.

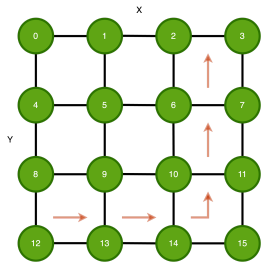

Figure 1a: X-Y DOR path

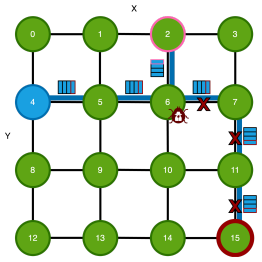

Figure 1b: HT misrouting

Still, its predictability makes it especially vulnerable to HT-induced misrouting. Figure 1b highlights this occurrence. Router 4 is the source router, and router 15 is the destination. Once the packet reaches infected router 6, the infected router changes the header flits (the pink flit), and the packet is routed north to router 2, which is in the opposite direction of router 15. From here, the DOR algorithm performs an X-Y routing path from router 2 to router 15 with increasing latency.

To address these vulnerabilities, we propose *Threat Obfuscation (re)Routing (ThOR)*, a pseudo-random routing mechanism that dynamically alters packet paths at runtime. By periodically deactivating routers and rerouting traffic adaptively, ThOR introduces controlled randomness that obfuscates packet trajectories, reducing repeated exposure to infected routers and enhancing resilience to HT attacks—all while maintaining reasonable performance and area overhead.

2. Implementation

In this section we elaborate upon the implementation details of ThOR. Specifically, ThOR is a modification to X-Y DOR, by enabling adaptive rerouting of packets through the network via pseudo-random behavior introduced in the network. By rerouting packets, we attempt to reduce the efficacy of HT impacts.

2.1 Pseudo-random number generation

To enable pseudo-random behavior in the network, we leverage a globally-accessed 64-bit linear feedback shift register (LFSR). We make use of an LFSR over other random number generators, as it has low hardware overhead and is composed of 64 flip-flops and 3 XOR gates, as can be seen in Figure 2a. Since the output of the LFSR contains 2^{64} possible values, we must truncate the output to a smaller N-bit vector, where N is sufficiently large that N covers the number of routers in the network. We then convert the N-bit vector into a one-hot vector by shuffling the N-bit output and passing it through simple boolean expressions. Since an LFSR by design has a deterministic sequence that iterates through each possible value before repeating, we utilize the aliasing afforded to us when truncating the output to generate a sequence that appears more random, and is harder for an attacker to determine the sequence.

Every clock cycle, each router accesses its representative bit from the LFSR's one-hot vector. If the bit is 1, the router disables the incoming links for this cycle by setting their edge weights to the maximum representable value, and requires neighboring routers to recalculate their routing table. If the bit is 0, the router will reactivate the incoming links, if they were previously disabled, by setting them to their default edge weight. Once again, neighboring routers will need to recalculate their routing table. In Figure 2a, router 2 is disabled, and is marked as such with the grey bubble. To maintain acceptable utilization of the links in the network, we must ensure that the LFSR shifts frequently enough that no link is disabled for too long.

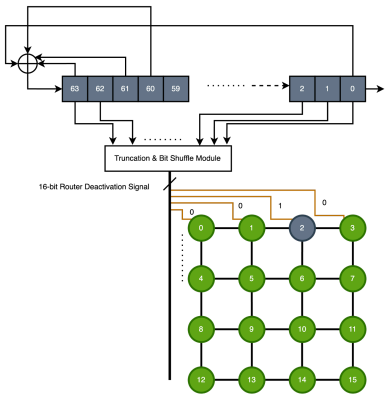

Figure 2a: LFSR Architecture

2.2 Routing past a disabled router

During most of a packet's traversal, assuming it does not interact with an infected router, it will assume a path abiding by X-Y DOR. If a packet is about to be routed through a disabled link according to DOR, the packet will be rerouted towards the direction of the destination router. In Figure 2b, a packet which is destined for router 15 is located in router 5. According to DOR, it would traverse to router 6, but in this cycle, router 6 is disabled. The candidate routers for rerouting are 1 and 9, but since 9 is closer to the destination, the packet is rerouted to router 9 instead. If a packet is attempting to turn in the middle of the network, it will continue forward, instead of making a turn. Note that in the cases along the mesh corners and edges, where a packet is turning, a U-turn may be necessary. After this rerouting step is finished, the packet continues its path assuming X-Y DOR, until it reaches the destination or encounters another disabled link.

An infected router's routing decision is mostly unchanged. It will still attempt to route away from the destination, but if the link it is misrouting through is disabled, it must choose a different path at random.

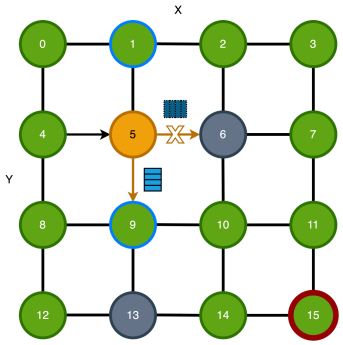

Figure 2b: 4-ary 2-cube Mesh ThOR Rerouting Decision

2.3 Deadlock concerns

Due to the nature of HTs, even in a network with a deadlock-free routing algorithm, we are unable to directly guarantee deadlock freedom, as the HT can hold on to necessary packets and can route them in arbitrary directions. Since deadlock may be unavoidable in the presence of HTs, we relaxed the constraints of deadlock-free routing, allowing our routers to reroute packets in whichever direction is necessary when encountering a disabled link.

3. Evaluation

We first experiment with how the infected routers affect the network performance in aspects of average flit latency and average packet latency without any router obfuscation from the LFSR deactivation mechanism. Second, we evaluate the overhead of our modified X-Y DOR, which solves no routes for

packets when routers are disabled by LFSR, in a network without infected routers. Finally, we evaluate the network performance with infected routers and integrate the LFSR deactivation mechanism into the network, assuming modified X-Y DOR is used (ThOR).

3.1 Simulated System and Methodology

We evaluate the effectiveness of ThOR under Garnet 2.0 [6] in gem5 [5]. We leverage Garnet network with the synthetic traffic for the ease of networking-only debugging and testing.

Network	
Simulation Cycles	100,000,000 cycles
Topology	4-ary 2-cube Mesh
Injected Traffic Type	Uniform Random
Injection Rate	0.01 - 0.9 (max is 1.0)
LFSR	
Shifting Latency	16 cycles
HT Routers	
Router Number	0, 1, 2, 4 out of 16
Router Latency	64 cycles
Misroute Probability	50%

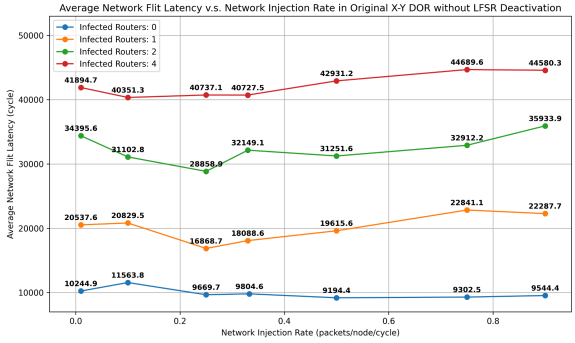

Figure 3a: Average Flit Latency in Baseline Network

3.2 Performance impact of infected routers

Two network metrics: average network flit latency and average network packet latency, are leveraged to measure the impact. As shown in Figure 3a, when no infected routers are present, all routers follow the X-Y DOR routing algorithm, and the average flit latency fluctuates around 10,000 cycles across all injection rates. However, the introduction of infected routers significantly degrades performance in terms of sharply increasing average network flit latency.

A similar trend is observed for average packet latency in Figure 3c, which increases with the number of infected routers. These results indicate that infected routers introduce severe network congestion, especially under heavy traffic, despite the simplicity of the mis-routing mechanism, the probabilistic nature of mis-routing, and their limited presence in the network.

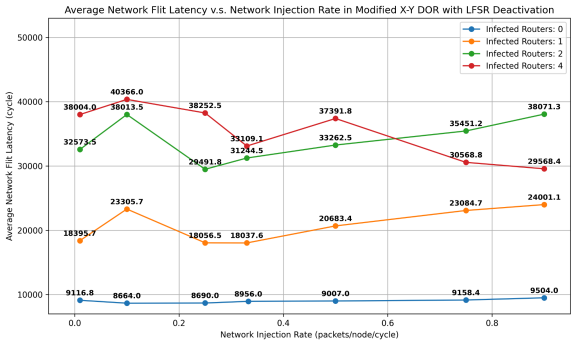

Figure 3b: Average Flit Latency in ThOR Network

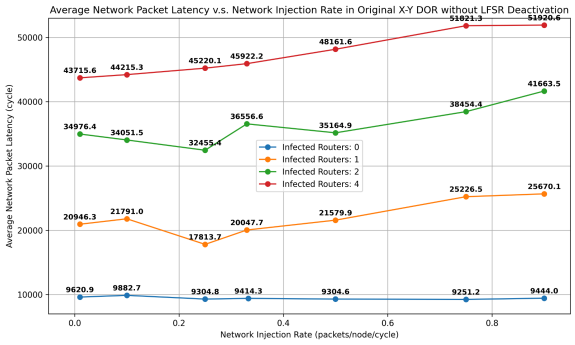

Figure 3c: Average Packet Latency in Baseline Network

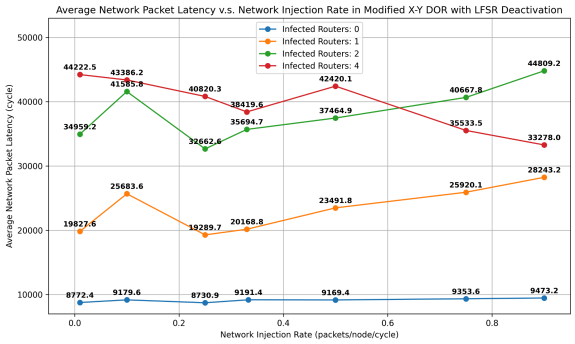

Figure 3d: Average Packet Latency in ThOR Network

3.3 Overhead of modified X-Y DOR

The performance degradation caused by infected routers highlights the need for fault-tolerant routing algorithms like ThOR, which mitigates their impact by using an LFSR to probabilistically deactivate a predefined number of routers. Every router has an approximately equal chance of being disabled, independent of whether it will serve as a next-hop in packet forwarding. To avoid situations where no valid route is returned due to router deactivation, the original X-Y DOR algorithm is modified. As described in Section 2.2, the modified X-Y DOR addresses this issue by redirecting packets to an

alternate network dimension when the preferred next-hop router is unavailable. The packet is then forwarded in the direction of the alternate network dimension closest to its destination. The overhead of the modified X-Y DOR algorithm is evaluated. In Figure 3a, the blue line shows the average flit latency with zero infected routers under the original X-Y DOR, while Figure 3b presents the same but under the modified X-Y DOR. The results indicate negligible overhead, with latencies nearly identical and occasionally lower in the modified version. A similar trend is observed for average packet latency in Figures 3c and 3d.

This performance benefit stems from the reduced link contention. In the unmodified X-Y DOR algorithm, the packets experience higher congestion on the X dimension. However, the modified version redirects the packet to the other network dimension if the next hop on the current dimension is disabled, which alleviates the link contention on the current routing dimension. Although this means that the free-of-deadlocks enforced by the base X-Y DOR is relaxed in our modified X-Y DOR, deadlocks are still tolerated in Garnet simulators and even in real NoCs. This suggests that ThOR can be integrated with any active, subactive [4], and reactive deadlock freedom routing mechanisms without sacrificing correctness.

3.4 Performance of ThOR

Compared to the baseline network, which employs the original X-Y DOR algorithm without LFSR-based router deactivation (Figure 3a), the ThOR network (Figure 3b), integrating both the modified X-Y DOR algorithm and LFSR, exhibits slightly higher average flit latencies when one or two infected routers are present in the network. This increase in latency is primarily due to the reduced number of active routers, which introduces moderate path inefficiency. A similar trend is observed for average packet latency when one or two infected routers are present in the network, as shown in Figures 3c and 3d.

However, when the number of infected routers increases to four, the ThOR network begins to outperform the baseline network. In Figure 3b, ThOR maintains average flit latencies below 40,000 cycles across all injection rates, while the baseline exceeds 40,000 cycles and even approaches 45,000 cycles under high injection rates in Figure 3a. At an injection rate of 0.9, ThOR achieves an average flit latency of approximately 29,568 cycles, significantly lower than the baseline's 44,580 cycles. Similar improvements are evident in average packet latency shown in Figures 3c and 3d. These results highlight ThOR's capability to mitigate the performance degradation caused by multiple infected routers.

4. Future Work

This report proposes a global LFSR to control router deactivation. However, broadcasting LFSR states becomes costly as the network scales, and managing synchronization across multiple frequency domains—especially under dynamic voltage and frequency scaling (DVFS)—adds further complexity. A potential solution is to adopt local LFSRs within routers. If deterministically seeded, local generators can ensure predictable deactivation patterns while maintaining sufficient runtime randomness to obscure which routers are inactive. Additionally, efforts to integrate random router activation with different routing algorithms, such as the link-weight-based look-up routing table in Garnet, revealed challenges. This algorithm relies on statically initialized tables based on link weights when the network is initiated. A modified scheme allowing real-time and on-demand look-up table reconstruction during route computation was implemented. Still, the approach proved impractical due to overheads and the lack of deadlock guarantees. Refinements are needed.

5. Conclusion

In this paper, Threat Obfuscation (re)Routing (ThOR), a pseudo-random routing algorithm, is proposed to mitigate the impacts of Hardware Trojans (HTs) in Networks-on-Chip (NoCs). By dynamically altering routing paths through probabilistic router deactivation and adaptive rerouting, ThOR reduces the predictability of packet trajectories and limits the influence of infected routers. Experimental results on a 4-ary 2-cube mesh in gem5 Garnet network demonstrate that while ThOR introduces minimal overhead under normal conditions, it significantly mitigates performance degradation in the presence of multiple infected routers, outperforming baseline routing algorithms at higher injection rates.

References

[1] M. Kumar J.Y.V, A. K. Swain, S. Kumar, S. R. Sahoo, and K. Mahapatra, “Run Time Mitigation of Performance Degradation Hardware Trojan Attacks in Network on Chip,” Jul. 2018, doi: https://doi.org/10.1109/isvlsi.2018.00139.

[2] S. Pasricha, J. Jose, and S. Deb, “Electronic, Wireless, and Photonic Network-on-Chip Security: Challenges and Countermeasures,” IEEE Design & Test, vol. 39, no. 6, pp. 90–98, Dec. 2022, doi: https://doi.org/10.1109/mdat.2022.3203017.

[3] R. Gupta, V. J. Kulkarni, J. Jose, and S. Nandi, “Securing On-chip Interconnect against Delay Trojan using Dynamic Adaptive Caging,” Proceedings of the Great Lakes Symposium on VLSI 2022, pp. 411–416, Jun. 2022, doi: https://doi.org/10.1145/3526241.3530333.

[4] M. Parasar, H. Farrokhbakht, N. Enright Jerger, P. V. Gratz, T. Krishna, and J. San Miguel, "DRAIN: Deadlock Removal for Arbitrary Irregular Networks," 2020 IEEE International Symposium on High Performance Computer Architecture (HPCA), San Diego, CA, USA, 2020, pp. 447-460, doi: 10.1109/HPCA47549.2020.00044.

[5] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti, R. Sen, K. Sewell, M. Shoaib, N. Vaish, M. D. Hill, and D. A. Wood, “The gem5 simulator,” SIGARCH Computer Architecture News, vol. 39, no. 2, pp. 1–7, 2011.

[6] N. Agarwal, T. Krishna, L. Peh, and N. K. Jha, “Garnet: A detailed on-chip network model inside a full-system simulator,” in 2009 IEEE International Symposium on Performance Analysis of Systems and Software, pp. 33–42, 2009.